instructions directly within the application.

Calibration procedure

1. Power on the ESP32 controller.
2. Connect the 5V supply to the PCA9685 after powering on the ESP32.
3. Connect the laptop, server, or Raspberry Pi 5 to the ESP32 Wi-Fi network named esp32_bera1.
4. Connect the USB camera to the processing computer.
5. Run basetable_alignment.py.
6. Place the target object at multiple locations across the worktable and record sufficient calibration points. These points establish the relationship between the camera's X-Y coordinates and Servo0 rotation.
7. After completing the base calibration, run reach_alignment.py.
8. Keep the base table fixed at 90° and place the target at multiple positions within the arm's reachable workspace. Record sufficient calibration points to calibrate the reach of Servo1 and Servo2.

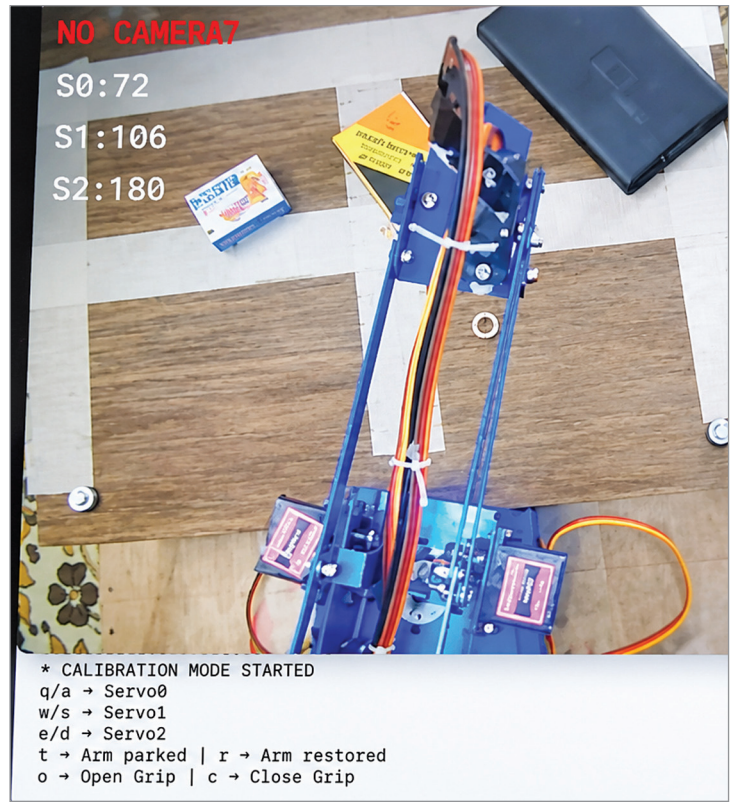


Fig. 5: Robotic arm during calibration and servo-position testing

Main program

After both calibration procedures are completed, transfer the collected calibration values to auto5.py, the main robotic control script. The system can then use the calibrated camera coordinates to determine the target position and calculate the required arm movements.

Software workflow

Camera→Python object detection→coordinate transformation/calibration→arm position calculation→ESP32→PCA9685→servo motors

Testing

After uploading the code to the ESP32 and properly assembling the system, the system is ready for testing.

For testing, run the following command:

python auto5.py matchbox

The system uses the overhead camera to detect and locate the matchbox, calculate the required arm position, and move the robotic arm to the target location. The arm then waits for further commands.

Fig. 5 shows the robotic arm during calibration and testing. The arm is positioned over the worktable, while the display shows the current servo positions: Servo0 = 72°, Servo1 = 106°, and Servo2 = 180°. The screen also indicates that no object is currently detected.

The lower part of the screen displays the calibration and manual-control options, including servo movement, arm restoration, and gripper open/close commands. These controls are used to verify servo operation, calibrate the arm's workspace, and ensure accurate positioning before running the main object-detection program.

The accuracy and repeatability of the system can be evaluated by placing the object at different positions across the worktable and observing the arm's performance. Additional calibration points can be added if finer positioning accuracy is required.

Additional Python codes

The following Python programs are also provided:

1. servo-control.py – Controls individual servos and allows them to be rotated to the required angular position

2. retract.py – Moves the robotic arm to its fully retracted position with the gripper closed

3. distance.py – Retrieves distance measurements from the ESP32 using the connected distance sensor

4. detect_all.py – Tests whether the trained object-detection model can successfully detect the target object EFY

TO DOWNLOAD SOURCE CODE AND RESOURCES

Scan this QR code or go to **https://sourcecode.electronicsforu.com** to access and download source code and other resources for this project

Somnath Bera (left) is an electronics enthusiast. He is a freelancer and has written several articles across the globe. Reon Samanta (right) is a digital electronics, PLC LLD, and automation enthusiast.